

# Palindrome Partitioning Logic

Absolutely. **Palindrome Partitioning II — LeetCode 132** is a very good problem for understanding **partition recursion + memoization**.

The key difficulty is understanding:

1. What does **start** mean?
2. What does **end** mean?
3. How are we making partitions?
4. What exactly are we memoizing?
5. Why does **dp[start]** work even though we have both **start** and **end**?
6. How does the recursion tree get reduced?

Let's build it from the ground up.

## 1. Problem in simple words

Given a string **s**, partition it into substrings such that **every substring is a palindrome**.

Return the **minimum number of cuts** required.

Example:

```
s = "aab"
```

Possible partition:

```
"a" | "a" | "b"
```

Cuts:

```
a  | a  | b
 ^   ^
2 cuts
```

Another:

```
"aa" | "b"
```

Cuts:

```
aa | b
 ^
1 cut
```

So answer:

```
1
```

## 2. The main idea: try every possible partition

Suppose we are currently at index **start**.

For:

```
a a b
0 1 2
```

Starting from **start = 0**, we can choose the ending position **end**.

So:

```
start = 0
```

Try:

```
end = 0 → "a"
end = 1 → "aa"
end = 2 → "aab"
```

We only make a partition if:

```
s[start...end]
```

is a palindrome.

So:

```
"a" → palindrome ✓
"aa" → palindrome ✓
"aab" → not palindrome ✗
```

Therefore we have two choices:

```
"a" | remaining "ab"
OR
"aa" | remaining "b"
```

This is the **partition logic**.

### 3. What does start mean?

This is extremely important.

**start** means:

**The index from which I still need to partition the string.**

For:

```
s = "aab"
```

Initially:

```
start = 0
```

because nothing has been processed.

If we choose:

```
"a"
```

then index 0 is finished.

So now:

```
start = 1
```

We need to partition:

```
"ab"
 ^
start
```

If we choose "aa":

```
aa | b
```

then:

```
start = 2
```

Now only:

```
"b"
 ^
start
```

is remaining.

### 4. What does end mean?

**end** is different.

**end** is used to **try different possible ending positions for the current partition**.

Suppose:

```
s = "aab"
```

and:

```
start = 0
```

Then:

```
end = 0
```

means:

```
s[0...0]
"a"
```

Then:

```
end = 1
```

means:

```
s[0...1]
"aa"
```

Then:

```
end = 2
```

means:

```
s[0...2]
"aab"
```

So you can think:

```
start = where my current piece begins
end   = where I decide my current piece ends
```

## 5. Visualizing start and end

Take:

```
s = a a b c
   0 1 2 3
```

If:

```
start = 1
```

we are here:

```
a | a b c
   ^
   start
```

Now **end** explores:

```
end = 1
a | [a] b c
   ^
   end
```

then:

```
end = 2
a | [a b] c
   ^
   end
```

then:

```
end = 3
a | [a b c]
   ^
   end
```

Every time:

```
s[start...end]
```

is a palindrome, we are allowed to make a partition there.

## 6. The recursive question

At every **start**, we ask:

What is the minimum number of cuts needed to partition the suffix starting from **start**?

Let's define:

```
C++
solve(start)
```

as:

Minimum cuts needed for **s[start ... n-1]**.

This is the most important definition.

For:

```
s = "aab"
```

we have:

```
solve(0)
```

meaning:

```
minimum cuts for "aab"
```

Then:

```
solve(1)
```

means:

```
minimum cuts for "ab"
```

And:

```
solve(2)
```

means:

```
minimum cuts for "b"
```

## 7. Where does end come in?

Inside **solve(start)**:

```
C++
for (int end = start; end < n; end++)
```

We try every possible first palindrome.

Conceptually:

```
solve(start)
|
|-- choose s[start...start]
|
|-- choose s[start...start+1]
|
|-- choose s[start...start+2]
|
|-- ...
```

For every palindrome:

```
s[start...end]
```

we make one cut and solve the remaining part:

```
1 + solve(end + 1)
```

Therefore:

```
C++
ans = min(ans, 1 + solve(end + 1));
```

## 8. Why +1?

Suppose:

```
"aa" | "b"
```

We made one partition:

```
aa | b
```

There is exactly:

```
1 cut
```

So:

```
C++
1 + solve(2)
```

But there's an important issue at the end.

If the entire remaining substring is already a palindrome, **we don't need another cut**.

For example:

```
"b"
```

is already a palindrome.

So:

```
"b"
```

requires:

```
0 cuts
```

Therefore:

```
C++
if (start == n)
    return 0;
```

## 9. Recursion tree for "aab"

Let's manually construct it.

```
a a b
0 1 2
```

Start:

```
solve(0)
```

Possible palindromes:

```
"a"
"aa"
```

So:

```
solve(0)
├── "a" → 1 + solve(1)
└── "aa" → 1 + solve(2)
```

Now expand `solve(1)`.

Remaining:

```
a b
```

Possible palindromes:

```
"a"
```

"ab" is not palindrome.

So:

```
solve(1)
└─ "a" → 1 + solve(2)
```

Now:

```
solve(2)
```

Remaining:

```
b
```

"b" is palindrome.

So:

```
solve(2)
→ 1 + solve(3)
```

And:

```
solve(3)
```

means we reached the end:

```
return 0
```

Therefore:

```
solve(2) = 1
```

Then:

```
solve(1) = 1 + solve(2)
          = 2
```

Now other branch:

```
solve(0)
└─ "aa"
    |
    → 1 + solve(2)
    → 1 + 1
    → 2
```

Wait — this gives 2, but the answer should be 1.

This reveals an important detail:

**The +1 approach above counts partitions, not cuts.**

If we make:

```
"aa" | "b"
```

there is one partition boundary, hence one cut.

But if we count the final "b" as another "piece selection", we'd get:

```
aa | b
^
1 cut
```

So a cleaner implementation either:

## Method A

Return **number of pieces**, then subtract 1.

or

## Method B

Return cuts directly and only add a cut when there is a remaining substring.

For learning recursion/memoization, I recommend **Method A first**, because it makes the partition logic extremely clear.

## 10. Method A — minimum number of palindrome pieces

Define:

```
C++
solve(start)
```

as:

Minimum number of palindrome substrings needed to cover  $s[start \dots n-1]$ .

Then:

```
C++
solve(start)
=
1 + solve(end + 1)
```

for every palindrome:

```
s[start...end]
```

At the end:

```
C++
solve(n) = 0
```

Finally:

```
C++
answer = solve(0) - 1;
```

Because:

```
number of pieces - 1 = number of cuts
```

Example:

```
aa | b
```

Number of pieces:

```
2
```

Number of cuts:

```
2 - 1 = 1
```

## 11. Code

```
C++
class Solution {
public:
    bool isPalindrome(string &s, int start, int end) {
        while (start < end) {
            if (s[start] != s[end])
                return false;

            start++;
            end--;
        }
        return true;
    }

    int solve(string &s, int start, vector<int> &dp) {
        // Nothing left
        if (start == s.size())
            return 0;

        // Already calculated
```

```

    if (dp[start] != -1)
        return dp[start];

    int ans = INT_MAX;

    // Try every possible ending point
    for (int end = start; end < s.size(); end++) {
        // Current substring must be palindrome
        if (isPalindrome(s, start, end)) {
            // Take this palindrome as one piece
            int pieces = 1 + solve(s, end + 1, dp);
            ans = min(ans, pieces);
        }
    }

    return dp[start] = ans;
}

int minCut(string s) {
    int n = s.size();
    vector<int> dp(n, -1);

    // Minimum palindrome pieces - 1 = minimum cuts
    return solve(s, 0, dp) - 1;
}
};

```

## 12. Now understand the dp

This is the part you specifically asked about: **why memoization works here**.

Our function is:

```

C++
solve(start)

```

Notice something:

Although we have an `end` variable, `end` is **not part of the state**.

Why?

Because `end` is just the current function's loop variable.

For a fixed:

```

start

```

we try **all possible** `end` values.

For example:

```

solve(1)

```

automatically does:

```

end = 1
end = 2
end = 3
...

```

Therefore we don't need:

```

C++
dp[start][end]

```

for this particular recursive formulation.

We only need:

```

C++
dp[start]

```

because:

```

solve(start)

```

always means the same thing:

minimum palindrome pieces required for the suffix beginning at `start`.



## 13. Very important distinction

You might think:

"But **start** and **end** both change, so shouldn't DP have two dimensions?"

Not necessarily.

Ask:

**What uniquely determines the future problem?**

After choosing:

```
s[start...end]
```

the remaining problem starts at:

```
end + 1
```

So the future is completely determined by:

```
end + 1
```

Once we're inside:

```
C++
solve(end + 1)
```

we don't care how we reached there.

For example:

```
solve(2)
```

could have been reached by:

```
"a" + "a"
```

or:

```
"aa"
```

But once we reach:

```
solve(2)
```

the remaining string is always:

```
"b..."
```

Therefore its answer is always the same.

That's exactly why memoization works.

## 14. The real power of memoization

Consider:

```
a a b c
0 1 2 3
```

There may be many ways to reach:

```
solve(2)
```

For example:

```
"a" + "a"
```

or:

```
"aa"
```

Both eventually ask:

```
What is the answer from index 2?
```

Without memoization:

```
solve(2)
```

would be calculated repeatedly.

With memoization:

```
C++
dp[2]
```

stores the answer.

First time:

```
solve(2)
↓
calculate
↓
dp[2] = result
```

Next time:

```
solve(2)
↓
dp[2] already exists
↓
return dp[2]
```

So recursion doesn't repeat the same work.

## 15. Complete dry run — "aab"

Let's do it very carefully.

String:

```
s = "aab"
index:
  0 1 2
  a a b
```

Initially:

```
dp = [-1, -1, -1]
```

Call:

```
C++
solve(0)
```

### Step 1: solve(0)

```
start = 0
```

Try every end.

**end = 0**

Substring:

```
s[0...0] = "a"
```

Palindrome:

```
YES
```

Take "a".

Remaining:

```
"ab"
 ^
start = 1
```

So:

```
C++
1 + solve(1)
```

## Step 2: solve(1)

Now:

```
a a b
 ^
start = 1
```

Try:

**end = 1**

Substring:

```
s[1...1] = "a"
```

Palindrome:

YES

Take "a".

Remaining:

```
"b"
 ^
start = 2
```

So:

```
C++
1 + solve(2)
```

## Step 3: solve(2)

Now:

```
a a b
 ^
start=2
```

Try:

**end = 2**

Substring:

```
"b"
```

Palindrome:

YES

So:

```
C++
1 + solve(3)
```

## Step 4: solve(3)

Now:

```
start = 3
n = 3
```

We reached the end:

```
C++
if (start == s.size())
    return 0;
```

Therefore:

```
solve(3) = 0
```

Go back to `solve(2)`:

```
1 + solve(3)
= 1 + 0
= 1
```

So:

```
dp[2] = 1
```

Our DP:

```
[-1, -1, 1]
```

## Step 5: Back to `solve(1)`

We had:

```
1 + solve(2)
```

And:

```
solve(2) = 1
```

Therefore:

```
1 + 1 = 2
```

So:

```
dp[1] = 2
```

DP:

```
[-1, 2, 1]
```

This means:

```
solve(1) = 2 pieces
```

Those pieces are:

```
"a" | "b"
```

## Step 6: Back to `solve(0)`

The first choice was:

```
"a"
```

So:

```
1 + solve(1)
```

=>

```
1 + 2
= 3
```

So currently:

```
ans = 3
```

This corresponds to:

```
"a" | "a" | "b"
```

3 pieces.

## Step 7: Continue solve(0)

Remember:

```
C++
for (int end = start; end < n; end++)
```

We haven't finished the loop.

Now:

```
end = 1
```

Substring:

```
s[0...1]
"aa"
```

Is palindrome?

```
YES
```

So choose:

```
"aa"
```

Remaining:

```
"b"
^
start = 2
```

Call:

```
C++
solve(2)
```

But wait!

We already calculated:

```
dp[2] = 1
```

Therefore:

```
C++
if (dp[start] != -1)
    return dp[start];
```

returns immediately:

```
solve(2) = 1
```

No recursion needed.

Now:

```
1 + solve(2)
= 1 + 1
= 2
```

So:

```
ans = min(3, 2)
= 2
```

## 16. What about end = 2?

Now:

```
s[0...2]
"aab"
```

Is it palindrome?

```
a != b
```

Therefore:

```
NOT palindrome
```

We don't make this partition.

So:

```
solve(0) = 2
```

And:

```
dp[0] = 2
```

Final DP:

```
[2, 2, 1]
```

Interpretation:

```
dp[0] = 2 pieces
dp[1] = 2 pieces
dp[2] = 1 piece
```

## 17. Convert pieces to cuts

We found:

```
2 pieces
```

The optimal partition is:

```
"aa" | "b"
```

Number of cuts:

```
pieces - 1
```

Therefore:

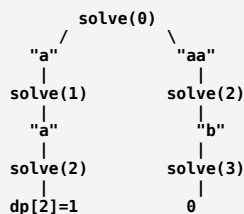
```
2 - 1 = 1
```

Answer:

```
1
```

## 18. The whole recursion visually

For "aab":



Notice something very important:

```
solve(2)
```

appears **twice**.

First path:

```
"a" → "a" → solve(2)
```

Second path:

```
"aa" → solve(2)
```

Without memoization:

```
solve(2)
```

would be recalculated.

With memoization:

```
dp[2] = 1
```

and the second call simply returns:

```
1
```

This is the core idea of DP.

## 19. Why the partition logic works

Think of the problem as:

```
START
|
v
Choose where the first palindrome ends
|
+---- "a" ----> solve(rest)
|
+---- "aa" ----> solve(rest)
|
+---- "aab" --> invalid
```

At every **start**, we are asking:

"What can I take as my first palindrome?"

Once we choose one:

```
s[start...end]
```

we don't need to worry about it anymore.

We move to:

```
end + 1
```

So:

```
[start ..... end] [end+1 ..... n-1]
  chosen           remaining
  palindrome      problem
```

This is the **partition pattern**.

## 20. One very important mental model

When you see code like:

```
C++
for (int end = start; end < n; end++) {
    if (isPalindrome(s, start, end)) {
        solve(end + 1);
    }
}
```

read it in English as:

"Starting from **start**, try every possible place where I can cut. If the substring from **start** to **end** is a palindrome, take it and recursively solve the remaining suffix."

That's the entire partition recursion.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

## 21. Why start is the DP state

Think of:

```
C++
solve(start)
```

as a **question**.

For example:

```
solve(0):
What is the minimum number of palindrome pieces for "aab"?

solve(1):
What is the minimum number of palindrome pieces for "ab"?

solve(2):
What is the minimum number of palindrome pieces for "b"?

solve(3):
What is the minimum number of palindrome pieces for ""?
```

Each **start** corresponds to one unique remaining suffix.

Therefore:

```
C++
dp[start]
```

stores the answer to that question.

## 22. But what if we used dp[start][end]?

You absolutely **can** use a 2D DP formulation, especially when precomputing palindrome status:

```
C++
palindrome[start][end]
```

This has a different meaning:

```
palindrome[i][j]
```

means:

Is  $s[i \dots j]$  a palindrome?

So there are actually two different states:

### Palindrome state

```
C++
palindrome[start][end]
```

because both boundaries matter.

### Partition state

```
C++
dp[start]
```

because once we choose the palindrome ending at **end**, the remaining problem starts at:

```
end + 1
```

This distinction is **very important**.

## 23. Production/efficient version

The previous code calls:

```
C++
isPalindrome(s, start, end)
```



inside recursion, which itself takes  $O(n)$ .

We can optimize palindrome checking by precomputing:

```
C++
pal[start][end]
```

Then palindrome checking becomes:

```
C++
if (pal[start][end])
```

in  $O(1)$ .

The solution becomes:

```
C++
class Solution {
public:
    int minCut(string s) {
        int n = s.size();
        // pal[i][j] = true if s[i...j] is palindrome
        vector<vector<bool>> pal(n, vector<bool>(n, false));

        for (int i = n - 1; i >= 0; i--) {
            for (int j = i; j < n; j++) {
                if (s[i] == s[j] &&
                    (j - i <= 2 || pal[i + 1][j - 1])) {
                    pal[i][j] = true;
                }
            }
        }

        vector<int> dp(n + 1, INT_MAX);
        dp[n] = 0;

        for (int start = n - 1; start >= 0; start--) {
            for (int end = start; end < n; end++) {
                if (pal[start][end]) {
                    dp[start] = min(
                        dp[start],
                        1 + dp[end + 1]
                    );
                }
            }
        }

        return dp[0] - 1;
    }
};
```

Complexity:

```
Palindrome table:  $O(n^2)$ 
Partition DP:       $O(n^2)$ 

Total:              $O(n^2)$ 
Space:              $O(n^2)$ 
```

## 24. The most important thing to remember

For this problem, keep these three ideas separate:

1. start  
↓  
Where the remaining problem begins.
2. end  
↓  
Where I try to end my current partition.
3. dp[start]  
↓  
Best answer for the entire suffix starting at start.

And the recursion is:

```
solve(start)
|
|-- try end = start
|-- try end = start + 1
|-- try end = start + 2
|-- ...
```

```

      |
      v
Is s[start...end] palindrome?
      |
      YES
      |
      v
1 + solve(end + 1)

```

Finally:

```

minimum palindrome pieces - 1
=
minimum number of cuts

```

## The single sentence to memorize

**At every start, try every end; if  $s[start \dots end]$  is a palindrome, take it as one partition and solve the remaining string from  $end + 1$ . Memoize the answer for each start.**

That is the core of **Palindrome Partitioning II**.

Yes — **you can absolutely use that style**, and in fact it lets you directly calculate the **number of cuts**, so you don't need the **pieces - 1** conversion.

The important idea is:

```

C++
if (start == n || isPalindrome(s, start, n - 1))
    return 0;

```

means:

If the remaining suffix is already a palindrome, we don't need any more cuts.

Let's write the recursion in that form.

## 1. Code using your base-case idea

```

C++
class Solution {
public:
    bool isPalindrome(string &s, int start, int end) {
        while (start < end) {
            if (s[start] != s[end])
                return false;

            start++;
            end--;
        }
        return true;
    }

    int solve(string &s, int start, vector<int> &dp) {
        int n = s.size();

        // No remaining string OR
        // the entire remaining string is already a palindrome
        if (start == n || isPalindrome(s, start, n - 1))
            return 0;

        // Already calculated
        if (dp[start] != -1)
            return dp[start];

        int ans = INT_MAX;

        // Try every possible partition point
        for (int end = start; end < n; end++) {
            // Take s[start...end] if it is a palindrome
            if (isPalindrome(s, start, end)) {
                // One cut after this palindrome
                int cuts = 1 + solve(s, end + 1, dp);
                ans = min(ans, cuts);
            }
        }
        return dp[start] = ans;
    }

    int minCut(string s) {
        int n = s.size();
        vector<int> dp(n, -1);
        return solve(s, 0, dp);
    }
};

```

```
}  
};
```

## 2. Why does this directly give cuts?

This time `solve(start)` means:

**Minimum number of cuts required to partition  $s[start \dots n-1]$  into palindromes.**

This is different from our previous definition.

Previously:

```
solve(start) = minimum palindrome pieces
```

Now:

```
solve(start) = minimum cuts
```

Therefore, when we choose:

```
s[start...end]
```

as our first palindrome, we do:

```
C++  
1 + solve(end + 1)
```

because we're making **one cut** after that palindrome.

But there's one special situation:

```
s[start...n-1]
```

itself is already a palindrome.

Then:

```
NO CUT IS REQUIRED
```

Therefore:

```
C++  
return 0;
```

That's exactly what your base condition handles.

## 3. Why is `isPalindrome(start, n - 1)`?

Suppose:

```
s = "aab"
```

and:

```
start = 2
```

Remaining string:

```
a a b  
  ^  
  start
```

The remaining substring is:

```
s[2...2] = "b"
```

It is already a palindrome.

Therefore:

```
C++  
isPalindrome(s, 2, 2)
```

returns:

```
true
```

and:

```
C++
return 0;
```

There is no reason to make a cut after "b".

## 4. Dry run — "aab"

Let's go very carefully.

```
s = "aab"
index:
  0 1 2
  a a b
```

Initial:

```
C++
solve(0)
```

We want:

```
minimum cuts for "aab"
```

### `solve(0)`

First check:

```
C++
start == n
```

No:

```
0 != 3
```

Then:

```
C++
isPalindrome(s, 0, 2)
```

Check:

```
"aab"
```

Not palindrome because:

```
a != b
```

So continue.

Now:

```
C++
for (int end = 0; end < 3; end++)
```

### `end = 0`

Current substring:

```
s[0...0]
"a"
```

Palindrome?

```
YES
```

So we take:

```
"a"
```

Partition:

```
a | ab
^
cut
```

Therefore:

```
C++
cuts = 1 + solve(1);
```

## 5. solve(1)

Remaining:

```
aab
^
start = 1
remaining = "ab"
```

Base condition:

```
C++
isPalindrome(s, 1, 2)
```

Check:

```
"ab"
```

Not palindrome.

So continue.

**end = 1**

Current substring:

```
s[1...1]
"a"
```

Palindrome:

```
YES
```

Take it:

```
a | b
^
```

One cut:

```
C++
1 + solve(2)
```

## 6. solve(2)

Remaining:

```
aab
^
start
remaining = "b"
```

Now our special base condition becomes important:

```
C++
isPalindrome(s, 2, 2)
```

"b" is a palindrome.

Therefore:

```
C++
return 0;
```

So:

```
solve(2) = 0
```

## 7. Go back to solve(1)

We had:

```
C++
1 + solve(2)
```

Therefore:

```
1 + 0 = 1
```

So:

```
solve(1) = 1
```

The partition is:

```
"a" | "b"
```

One cut.

## 8. Go back to solve(0)

For the first choice:

```
"a"
```

we had:

```
C++
1 + solve(1)
```

Therefore:

```
1 + 1
= 2
```

So currently:

```
ans = 2
```

This represents:

```
"a" | "a" | "b"
```

There are:

```
2 cuts
```

## 9. Continue the loop at solve(0)

We haven't finished trying `end`.

Now:

```
C++
end = 1
```

Current substring:

```
s[0...1]
"aa"
```

Palindrome?

YES

So choose:

"aa"

Partition:

```
aa | b
  ^
  cut
```

We do:

```
C++
1 + solve(2)
```

But we already know:

```
solve(2) = 0
```

because "b" is a palindrome.

Therefore:

```
1 + 0 = 1
```

So:

```
ans = min(2, 1)
      = 1
```

## 10. end = 2

Now:

```
s[0...2]
"aab"
```

Not palindrome.

So nothing happens.

Finally:

```
solve(0) = 1
```

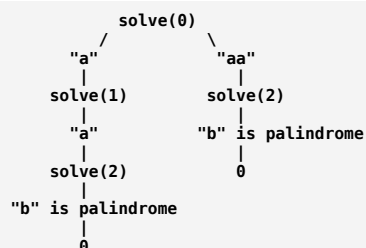
Therefore:

```
answer = 1
```

Correct.

## 11. Recursion tree

The whole thing looks like this:



Calculate the branches:

```
"a" → "a" → "b"
1 + 1 + 0
  ↓
  2 cuts
```

and:

```
"aa" → "b"
1 + 0
  ↓
  1 cut
```

Take minimum:

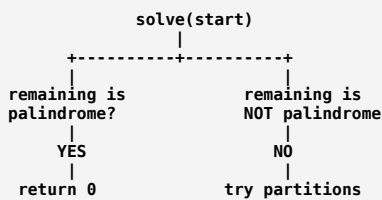
```
min(2, 1)
= 1
```

## 12. Why the base case is so useful

This:

```
C++
if (start == n || isPalindrome(s, start, n - 1))
    return 0;
```

basically says:



For example:

```
"aab"
```

At:

```
start = 0
```

remaining:

```
"aab"
```

not palindrome → continue.

At:

```
start = 1
```

remaining:

```
"ab"
```

not palindrome → continue.

At:

```
start = 2
```

remaining:

```
"b"
```

palindrome → **return 0 immediately**.

This is a very natural way to think about the recursion.

## 13. One subtle point about the base case

You wrote:

```
C++
if (start == n || isPalindrome(s, start, n - 1))
    return 0;
```



This is correct.

But notice:

```
C++
start == n
```

already means:

```
there is nothing left
```

while:

```
C++
isPalindrome(s, start, n - 1)
```

means:

```
there is something left,
but the entire remaining substring is already a palindrome.
```

Both mean:

```
NO MORE CUTS ARE REQUIRED
```

So:

```
C++
return 0;
```

is correct in both cases.

## 14. One important optimization issue

Your version is logically correct, but there is an important performance consideration.

You are doing:

```
C++
isPalindrome(s, start, n - 1)
```

at every recursive state.

And then again:

```
C++
isPalindrome(s, start, end)
```

inside the loop.

`isPalindrome()` itself is  $O(n)$ .

So this version can become more expensive than necessary.

For LeetCode, a more optimized version precomputes palindrome information:

```
pal[i][j]
```

and then both checks become  $O(1)$ .

But **conceptually**, your version is completely valid and is actually a very good way to understand the recursive partition logic.

## 15. The key difference between the two approaches

### Previous approach

```
C++
solve(start)
= minimum number of palindrome pieces
```

Therefore:

```
C++
return solve(0) - 1;
```

because:

```
pieces - 1 = cuts
```

## Your approach

```
C++
solve(start)
= minimum number of cuts
```

Therefore:

```
C++
return solve(0);
```

No subtraction is required.

The key is the base case:

```
C++
if (isPalindrome(s, start, n - 1))
    return 0;
```

because:

If everything remaining can already be one palindrome, we're done and need zero more cuts.

## The mental model I would use

When writing this recursion, think:

```
solve(start)

"What is the minimum number of cuts needed
for the remaining string?"

    |
    ↓
Try every end

    |
    ↓
Is s[start...end] palindrome?

    |
    YES
    |
    ↓
Take it as the current partition

    |
    ↓
1 cut + solve(end + 1)

    |
    ↓
Take minimum
```

Except when:

```
remaining substring itself is palindrome
```

then:

```
0 cuts
```

That gives you exactly the version you proposed.

Exactly. Now we can take the **memoized version you just understood** and convert it into **bottom-up tabulation**.

The most important thing is not to memorize the tabulation code. We will mechanically convert the recursive state into a table.

## 1. First, recall our memoized state

Our recursive code was based on:

```
C++
solve(start)
```

where:

$\text{solve}(\text{start})$  = minimum number of cuts required to partition  $s[\text{start} \dots n-1]$  into palindromes.

The recurrence was:

```
C++
if (start == n || isPalindrome(s, start, n - 1))
    return 0;
```

Otherwise:

```
C++
for (int end = start; end < n; end++) {
    if (isPalindrome(s, start, end)) {
        int cuts = 1 + solve(end + 1);
        ans = min(ans, cuts);
    }
}
```

So mathematically:

```
dp[start] = min(
    1 + dp[end + 1]
)
```

for every  $\text{end}$  such that:

```
s[start...end] is palindrome
```

## 2. First step in converting recursion $\rightarrow$ tabulation

The first question in every recursion-to-tabulation conversion is:

**What does my recursive state represent?**

Here:

```
C++
solve(start)
```

becomes:

```
C++
dp[start]
```

So:

```
solve(start)  $\rightarrow$  dp[start]
```

And the meaning stays exactly the same:

```
dp[start]
=
minimum cuts required for s[start...n-1]
```

This is the most important step.

## 3. Find the base case

Our recursive base case was:

```
C++
if (start == n)
    return 0;
```

Therefore in tabulation:

```
C++
```

```
dp[n] = 0;
```

Why?

Because:

```
start = n
```

means:

```
a a b ^
```

There is nothing left.

Therefore:

```
0 more cuts
```

So:

```
C++
dp[n] = 0;
```

## 4. What about this base condition?

Our recursive code also had:

```
C++
if (isPalindrome(s, start, n - 1))
    return 0;
```

This means:

If the entire remaining suffix is already a palindrome, no cut is required.

In tabulation, we don't necessarily need to explicitly write:

```
C++
if (isPalindrome(...))
    dp[start] = 0;
```

because the recurrence itself will naturally produce 0 if we handle the **whole remaining substring** as a palindrome.

But let's see how.

Suppose:

```
s = "aab"
```

At:

```
start = 2
```

remaining:

```
"b"
```

When we try:

```
end = 2
```

we have:

```
s[2...2] = "b"
```

which is palindrome.

Then:

```
1 + dp[end + 1]
```

becomes:

```
1 + dp[3]
```

which is:

```
1 + 0 = 1
```

But we want:

```
dp[2] = 0
```

So **we cannot blindly use** `1 + dp[end+1]` **for the final palindrome.**

This is the key difference.

## 5. How do we represent "no cut if the whole suffix is palindrome"?

We can write:

```
C++
if (pal[start][n - 1])
    dp[start] = 0;
```

Otherwise:

```
C++
dp[start] = min(
    1 + dp[end + 1]
)
```

for valid palindrome partitions.

So our tabulation becomes:

```
if entire suffix is palindrome:
    dp[start] = 0
else:
    try every palindrome prefix
    dp[start] = min(1 + dp[end+1])
```

This exactly matches the recursive logic.

## 6. But first we need `pal[start][end]`

Our recursion repeatedly asks:

```
C++
isPalindrome(s, start, end)
```

For tabulation, it is much better to precompute this.

Define:

```
C++
pal[i][j]
```

as:

`true` if `s[i...j]` is a palindrome.

For example:

```
s = "aab"
```

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | T | T | F |
| 1 |   | T | F |
| 2 |   |   | T |

Because:

```
"a"  → palindrome
"aa" → palindrome
"aab" → not
"a"  → palindrome
"ab" → not
"b"  → palindrome
```

## 7. How do we calculate `pal[i][j]`?

A string is palindrome when:

```
first character == last character
```

and the inside is also palindrome.

So:

```
C++
s[i] == s[j] &&
pal[i + 1][j - 1]
```

There are small-length exceptions.

The standard condition is:

```
C++
if (s[i] == s[j] &&
    (j - i <= 2 || pal[i + 1][j - 1]))
```

Why  $j - i \leq 2$ ?

### Length 1

```
"a"
```

Obviously palindrome.

### Length 2

```
"aa"
```

If both characters are equal, palindrome.

### Length 3

```
"aba"
```

Only the middle character exists, so we don't need to check anything inside.

Therefore:

```
C++
j - i <= 2
```

handles lengths 1, 2 and 3.

## 8. Why do we calculate $i$ backwards?

This is an important DP dependency.

For:

```
C++
pal[i][j]
```

we need:

```
C++
pal[i + 1][j - 1]
```

Therefore:

```
i + 1
```

must already be calculated.

So we go:

```
i = n - 1 → 0
```

Example:

```
i = 2
i = 1
```

```
i = 0
```

This ensures the inside substring has already been calculated.

## 9. Now the actual cut DP

Once `pal` is ready, define:

```
C++
dp[i]
```

as:

```
minimum cuts for s[i...n-1]
```

We know:

```
C++
dp[n] = 0;
```

Now suppose:

```
i = 1
```

and:

```
s = "aab"
```

Remaining:

```
"ab"
```

Try every `j`:

```
j = 1 → "a"
j = 2 → "ab"
```

Only `"a"` is palindrome.

Therefore:

```
dp[1] = 1 + dp[2]
```

## 10. Why do we fill `dp` backwards?

Look at our recurrence:

```
C++
dp[start] = 1 + dp[end + 1]
```

Notice:

```
end + 1 > start
```

So `dp[start]` depends on a **larger index**.

For example:

```
dp[0] depends on dp[1], dp[2], ...
dp[1] depends on dp[2], dp[3], ...
dp[2] depends on dp[3]
```

Therefore we must calculate:

```
dp[n]
dp[n-1]
dp[n-2]
...
dp[0]
```

So:

```
C++
for (int start = n - 1; start >= 0; start--)
```

## 11. The complete conversion

Recursive:

```
solve(start)
```

becomes:

```
dp[start]
```

Recursive base:

```
C++
start == n → return 0
```

becomes:

```
C++
dp[n] = 0;
```

Recursive:

```
C++
solve(end + 1)
```

becomes:

```
C++
dp[end + 1]
```

Recursive:

```
C++
1 + solve(end + 1)
```

becomes:

```
C++
1 + dp[end + 1]
```

Recursive:

```
C++
ans = min(ans, ...)
```

becomes:

```
C++
dp[start] = min(dp[start], ...)
```

And recursion order:

```
start → end+1
```

becomes reverse iteration:

```
n-1 → 0
```

## 12. Complete tabulation code

```
C++
class Solution {
public:
    int minCut(string s) {
        int n = s.size();

        // -----
        // Step 1: Precompute palindrome
        // -----

        vector<vector<bool>> pal(
            n,
            vector<bool>(n, false)
        );
```



```

for (int i = n - 1; i >= 0; i--) {
    for (int j = i; j < n; j++) {
        if (s[i] == s[j] &&
            (j - i <= 2 || pal[i + 1][j - 1])) {
            pal[i][j] = true;
        }
    }
}

// -----
// Step 2: DP for minimum cuts
// -----

vector<int> dp(n + 1, INT_MAX);

// Base case:
// no string remaining = 0 cuts
dp[n] = 0;

// Fill from right to left
for (int start = n - 1; start >= 0; start--) {
    // Entire remaining string is palindrome
    if (pal[start][n - 1]) {
        dp[start] = 0;
        continue;
    }

    // Try every possible partition
    for (int end = start; end < n; end++) {
        if (pal[start][end]) {
            dp[start] = min(
                dp[start],
                1 + dp[end + 1]
            );
        }
    }
}

return dp[0];
};

```

## 13. Now let's dry run it

Take:

```
s = "aab"
```

Indexes:

```

0  1  2
a  a  b

```

## Step 1: Build palindrome table

We calculate:

```
pal[i][j]
```

The final table is conceptually:

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | T | T | F |
| 1 |   | T | F |
| 2 |   |   | T |

Meaning:

```

pal[0][0] = true   → "a"
pal[0][1] = true   → "aa"
pal[0][2] = false  → "aab"

pal[1][1] = true   → "a"
pal[1][2] = false  → "ab"

pal[2][2] = true   → "b"

```

## 14. Initialize cut DP

We create:

```
C++
vector<int> dp(n + 1, INT_MAX);
```

Initially:

```
dp = [∞, ∞, ∞, ∞]
```

Then:

```
C++
dp[3] = 0;
```

because index 3 means:

```
a a b
    ^
  nothing remaining
```

So:

```
dp = [∞, ∞, ∞, 0]
```

## 15. Calculate dp[2]

Now:

```
start = 2
```

Remaining:

```
"b"
```

First:

```
C++
pal[2][2]
```

is:

```
true
```

Therefore:

```
C++
dp[2] = 0;
```

Because "b" itself is already a palindrome.

Now:

```
dp = [∞, ∞, 0, 0]
```

This corresponds exactly to our recursive:

```
C++
if (isPalindrome(s, start, n - 1))
    return 0;
```

## 16. Calculate dp[1]

Now:

```
start = 1
```

Remaining:

```
"ab"
```

First check:

```
C++
```

```
pal[1][2]
```

which is:

```
false
```

So we cannot return 0.

Now try every end.

```
end = 1
```

```
s[1...1] = "a"
```

Palindrome:

```
YES
```

Therefore:

```
C++
dp[1] = min(
    dp[1],
    1 + dp[2]
);
```

We know:

```
dp[2] = 0
```

So:

```
dp[1] = 1 + 0
       = 1
```

```
end = 2
```

```
s[1...2] = "ab"
```

Not palindrome.

Ignore it.

Therefore:

```
dp[1] = 1
```

DP now:

```
dp = [∞, 1, 0, 0]
```

## 17. Calculate dp[0]

Now:

```
start = 0
```

Remaining:

```
"aab"
```

Check:

```
C++
pal[0][2]
```

It's:

```
false
```

So we need to partition.

```
end = 0
```

```
s[0...0] = "a"
```

Palindrome.

Therefore:

```
C++
dp[0] = min(
    dp[0],
    1 + dp[1]
);
```

We know:

```
dp[1] = 1
```

Therefore:

```
dp[0] = 2
```

Partition represented:

```
"a" | "ab"
```

But "ab" isn't palindrome, so the recursion inside `dp[1]` further partitions it:

```
"a" | "a" | "b"
```

Total:

```
2 cuts
```

**end = 1**

Now:

```
s[0...1] = "aa"
```

Palindrome.

Therefore:

```
C++
dp[0] = min(
    2,
    1 + dp[2]
);
```

And:

```
dp[2] = 0
```

So:

```
1 + 0 = 1
```

Therefore:

```
dp[0] = min(2, 1)
        = 1
```

Partition:

```
aa | b
  ^
  1 cut
```

**end = 2**

```
s[0...2] = "aab"
```

Not palindrome.

Ignore.

Final:

```
dp[0] = 1
```

Return:

```
C++
return dp[0];
```

Answer:

```
1
```

## 18. The entire DP table evolution

For:

```
s = "aab"
```

we start:

```
dp = [∞, ∞, ∞, 0]
```

Then:

```
start = 2
dp = [∞, ∞, 0, 0]
```

Then:

```
start = 1
dp = [∞, 1, 0, 0]
```

Then:

```
start = 0
dp = [1, 1, 0, 0]
```

Final:

```
      0  1  2  3
dp = [1, 1, 0, 0]
```

Interpretation:

```
dp[0] = minimum cuts for "aab" = 1
dp[1] = minimum cuts for "ab"  = 1
dp[2] = minimum cuts for "b"   = 0
dp[3] = minimum cuts for ""    = 0
```

## 19. See the direct connection with recursion

This is probably the most important part.

### Memoization

You had:

```
C++
int solve(string &s, int start, vector<int> &dp) {
    if (start == n || isPalindrome(s, start, n - 1))
        return 0;

    if (dp[start] != -1)
        return dp[start];

    int ans = INT_MAX;
    for (int end = start; end < n; end++) {
        if (isPalindrome(s, start, end)) {
            int cuts = 1 + solve(s, end + 1, dp);
            ans = min(ans, cuts);
        }
    }
}
```

```
    return dp[start] = ans;
}
```

## Tabulation

We remove recursion and explicitly calculate the states:

```
C++
dp[n] = 0;
for (int start = n - 1; start >= 0; start--) {
    if (pal[start][n - 1]) {
        dp[start] = 0;
        continue;
    }
    for (int end = start; end < n; end++) {
        if (pal[start][end]) {
            dp[start] = min(
                dp[start],
                1 + dp[end + 1]
            );
        }
    }
}
```

The mapping is:

| Memoization                         | Tabulation                        |
|-------------------------------------|-----------------------------------|
| <code>solve(start)</code>           | <code>dp[start]</code>            |
| <code>return 0 at start == n</code> | <code>dp[n] = 0</code>            |
| <code>solve(end + 1)</code>         | <code>dp[end + 1]</code>          |
| recursive call                      | table lookup                      |
| <code>ans = min(...)</code>         | <code>dp[start] = min(...)</code> |
| recursion naturally moves right     | loop goes <code>n-1 → 0</code>    |
| <code>isPalindrome()</code>         | <code>pal[start][end]</code>      |

## 20. Why start goes backwards

This is something you should recognize in DP problems.

Our dependency is:

```
dp[start]
  ↓
dp[end + 1]
```

Since:

```
end >= start
```

we have:

```
end + 1 > start
```

Therefore:

```
dp[0] depends on dp[1], dp[2], dp[3]...
dp[1] depends on dp[2], dp[3]...
dp[2] depends on dp[3]
```

So we must calculate:

```
dp[3]
  ↓
dp[2]
  ↓
dp[1]
  ↓
dp[0]
```

This is exactly why:

```
C++
for (int start = n - 1; start >= 0; start--)
```

## 21. One subtle improvement

The code above uses:

```
C++
if (pal[start][n - 1])
    dp[start] = 0;
```

This is the direct tabulation equivalent of your recursive base case.

But we could also write the DP in a slightly cleaner way by initializing:

```
C++
dp[start] = n;
```

and treating the whole palindrome specially:

```
C++
if (pal[start][end]) {
    if (end == n - 1)
        dp[start] = 0;
    else
        dp[start] = min(dp[start], 1 + dp[end + 1]);
}
```

However, **I prefer the first version** because it mirrors your recursive code exactly:

```
C++
if (pal[start][n - 1])
    dp[start] = 0;
```

Then try partitions only when necessary.

## 22. Complexity

We have two DP components.

### Palindrome table

There are:

$O(n^2)$

possible (i, j) pairs.

Each is calculated in  $O(1)$ .

Therefore:

$O(n^2)$

### Cut DP

For every start:

$O(n)$

possible end.

There are:

$O(n)$

starts.

Therefore:

$O(n^2)$

Total:

Time =  $O(n^2)$   
Space =  $O(n^2)$

## 23. The conversion pattern you should learn

For this problem, don't just memorize the final tabulation. Learn this conversion:

#### RECURSION

```
solve(start)
  ↓
try every end
  ↓
if palindrome
  ↓
1 + solve(end + 1)
```

becomes:

#### TABULATION

```
dp[start]
  ↓
try every end
  ↓
if palindrome
  ↓
1 + dp[end + 1]
```

And because:

```
dp[start] depends on dp[end + 1]
```

we fill:

```
n → 0
```

So the complete mental process is:

1. Identify recursive state
 

```
solve(start)
      ↓
```
2. Convert state to DP
 

```
dp[start]
      ↓
```
3. Find base case
 

```
dp[n] = 0
      ↓
```
4. Look at recursive dependency
 

```
solve(end + 1)
      ↓
```
5. Convert it
 

```
dp[end + 1]
      ↓
```
6. Determine direction
 

```
end + 1 > start
      ↓
```
7. Therefore fill backwards
 

```
start = n-1 → 0
      ↓
```

That's the actual process of converting this particular memoization solution into tabulation.

Yes. You want the **tabulation version**, but instead of using a precomputed `pal[][]` table, you want to directly call:

```
C++
isPalindrome(s, start, end)
```

which returns `true` or `false`.

Here is the clean final code.

```
C++
class Solution {
public:
    // Returns true if s[start...end] is a palindrome
    bool isPalindrome(string &s, int start, int end) {
        while (start < end) {
            if (s[start] != s[end])
                return false;
            start++;
            end--;
        }
    }
};
```



```

    }
    return true;
}

int minCut(string s) {
    int n = s.size();
    // dp[i] = minimum cuts required
    // to partition s[i...n-1]
    vector<int> dp(n + 1, 0);

    // dp[n] = 0
    // Nothing remains, so no cuts needed.

    // Calculate from right to left
    for (int start = n - 1; start >= 0; start--) {
        // If the entire remaining string is palindrome,
        // no cut is required.
        if (isPalindrome(s, start, n - 1)) {
            dp[start] = 0;
            continue;
        }

        int ans = INT_MAX;

        // Try every possible ending position
        for (int end = start; end < n; end++) {
            // Current part s[start...end] must be palindrome
            if (isPalindrome(s, start, end)) {

                // One cut after this palindrome
                // + minimum cuts for remaining part
                int cuts = 1 + dp[end + 1];

                ans = min(ans, cuts);
            }
        }
        dp[start] = ans;
    }
    return dp[0];
}
};

```

## The important part

The recursive version:

```

C++
int cuts = 1 + solve(s, end + 1, dp);

```

becomes in tabulation:

```

C++
int cuts = 1 + dp[end + 1];

```

because `solve(end + 1)` has already been calculated.

And:

```

C++
if (isPalindrome(s, start, n - 1))
    return 0;

```

becomes:

```

C++
if (isPalindrome(s, start, n - 1)) {
    dp[start] = 0;
    continue;
}

```

So the complete flow is:

```

start = n-1
↓
start = n-2
↓
start = n-3
↓
...
↓
start = 0

```

For each `start`:

```

try every end
  ↓
is s[start...end] palindrome?
  ↓
YES
  ↓
1 + dp[end + 1]
  ↓
take minimum

```

For "aab":

```

dp[3] = 0
dp[2] = 0      // "b" is palindrome
dp[1] = 1      // "a" | "b"
dp[0] = 1      // "aa" | "b"
answer = dp[0] = 1

```

**One caveat:** this version is easy to understand, but `isPalindrome()` is called many times, so the worst-case time complexity is  $O(n^3)$ . The `pal[][]` precomputation version reduces it to  $O(n^2)$ .

ChatGPT can make mistakes. Check important info.